



COMPUTAÇÃO PARALELA

AULA 3



Prof. Leonardo Gomes



CONVERSA INICIAL

Discutiremos aqui o *software* como o sistema operacional que administra a execução de diversos programas, tirando máximo proveito de múltiplos processadores. Conversamos também sobre como o programador pode pensar seus códigos de forma paralela, além dos principais problemas e soluções. Ao final desta aula esperamos atingir os seguintes objetivos, que serão avaliados ao longo da disciplina:

1. Definição de Programa, processo e *thread*, e suas diferenças;
2. Escalonamento de processos e threads pelo sistema operacional;
3. Conceitos básicos e primeiros programas paralelos, utilizando várias threads;
4. Estudo sobre alguns dos problemas clássicos de sincronização.

TEMA 1 – DIFERENÇAS ENTRE PROGRAMA, PROCESSO E THREAD

Programas, processos e *thread* são três conceitos fundamentais e que frequentemente acabam sendo confundidos. Compreender suas propriedades e diferenças é muito importante para o entendimento das vantagens e aplicações do paralelismo.

1.1 Programa

O programa é um arquivo executável composto de uma sequência de instruções codificadas por um programador com um propósito específico visando ser executado pelo computador.

O programa fica alocado em disco ou algum outro tipo de memória secundária. No momento da execução ele é carregado para a memória principal pelo *kernel* do sistema operacional.

Os *softwares* que lidamos no dia a dia, como os navegadores de internet, editores de texto e *media players* são exemplos de executáveis (programas). O próprio *kernel* do sistema operacional também é um programa. Lembrando que *kernel* é um dos primeiros programas a serem executados pelo sistema operacional no momento que o sistema é ligado: ele é responsável por carregar e gerenciar a execução dos demais programas na memória.



1.2 Processo

Os processos são instâncias de um programa rodando na memória principal de um computador. Enquanto o programa em si é em geral estático e reside em uma memória secundária, o processo seria a sequência de instruções carregadas e ativamente executadas. Um mesmo programa pode possuir diversos processos; por exemplo, pense no seu editor de texto. O usuário tem a liberdade de executá-los várias vezes, e todas essas instâncias estarão em funcionamento simultaneamente, cada uma podendo ter um texto diferente. Uma vez que o sistema operacional seja reiniciado, os processos são encerrados, mas o programa continua salvo em memória.

As bibliotecas dinâmicas (DLL, do inglês *Dynamic Link Library*), que são implementadas por alguns sistemas operacionais como o *Windows*, são processos que carregam algumas funcionalidades comuns em muitos programas, por exemplo, uma biblioteca de instruções gráficas. É uma forma de otimizar recursos. Suponha que 20 processos executem um mesmo kit de funcionalidades gráficas em comum rodando ao mesmo tempo. Se essa biblioteca estiver em uma DLL, evita-se carregá-la vinte vezes na memória.

O sistema operacional controla todos os processos, desde o tempo de acesso ao processador e nível de prioridade sobre o mesmo que cada um terá. Controla também quanto de memória e quais posições os processos terão disponíveis, além das permissões de acesso a outros recursos que ficam vinculados ao processo como um todo.

Os sistemas operacionais oferecem para cada processo um largo espaço contínuo de endereçamento de memória virtual. Essa técnica oferece a ilusão de que cada processo roda sozinho no computador. Naturalmente, isso não é verdade, porém esse paradigma tira do programador o difícil trabalho de gerenciar a memória física. O sistema operacional cuida de alocar na memória todos os dados e instruções em execução, e caso a memória principal não seja suficiente para comportar tudo, utiliza um espaço da memória principal para essa finalidade, chamada também de memória *swap*.

1.3 Thread

A *thread* seria cada sequência única de instruções executadas por um processo. Voltando ao exemplo do editor de texto, podemos pensar em uma



thread sendo responsável por monitorar o teclado e ler o texto digitado, enquanto outra *thread* simultaneamente exibe o texto digitado na tela e todos os elementos visuais do programa. Uma terceira *thread* é responsável por gerenciar a impressão do texto.

O sistema operacional trata de forma igual *threads* e processos no que diz respeito à alocação de tempo de processamento. Mas em termos de memória, é responsabilidade da aplicação controlar o acesso compartilhado por todo o processo.

Pensando em um sistema com vários núcleos de processamento, ele será capaz de operar simultaneamente várias *threads* de um mesmo ou de diversos processos.

É possível ter um programa que faça a própria gestão de suas *threads* por meio de bibliotecas próprias. Essa gestão, no entanto, geralmente acaba não contando com as vantagens de executar em múltiplos núcleos de processamento. Esse conceito é conhecido por *soft*, *light* ou *green thread*.

1.4 Diferença

Na Figura 1 é ilustrada a relação entre programa, processo e *thread*, sendo o programa um superconjunto de processos que, por sua vez, é um superconjunto de *threads*.

Quadro 2 – Relação entre processo e *thread*

Processo	Thread
Computacionalmente caro para instanciar um novo processo	Computacionalmente barato para instanciar uma nova <i>thread</i>
Cada processo possui seu próprio espaço de memória	Cada <i>thread</i> utiliza a memória do processo ao qual pertence
Comunicação entre processos é lenta, pois cada processo possui um endereçamento próprio	Comunicação entre <i>threads</i> de um mesmo processo é rápida, por compartilharem o mesmo espaço de memória e de endereçamento
A troca de contexto entre processos é pesada do ponto de vista do processador	A troca de contexto entre diferentes <i>threads</i> é bastante otimizada pelo processador

Por troca de contexto dentro de sistemas operacionais, entendemos como sendo a tarefa computacional de substituir quais processos estão sendo executados pelo computador em um dado momento. Suponha que um

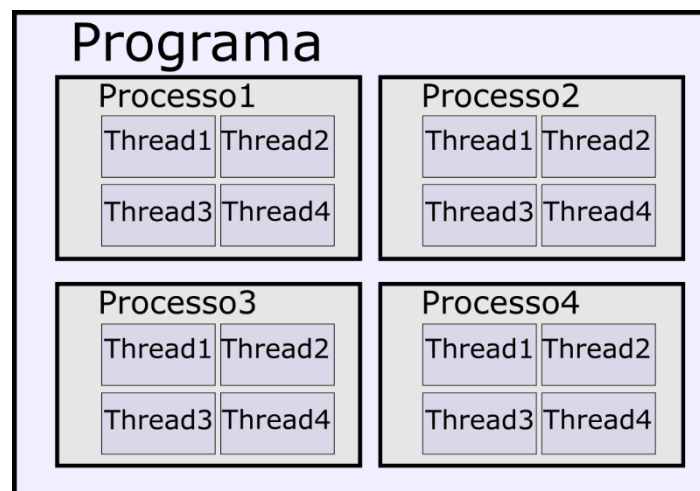


computador esteja executando 50 processos, mas conte com apenas quatro processadores. Os 50 processos terão que revezar o uso desses processadores a cada intervalo de tempo. Quando um processo sai de execução, vai para uma fila e aguarda sua vez de ser chamado para execução novamente. O sistema operacional pode dividir o tempo do processador de forma igualitária ou dar maior prioridade para determinados processos. Em todo caso, no momento que um processo deixa de estar em execução e passa para essa fila é necessário executar uma troca de contexto.

Um processo possui um contexto de *hardware* que seria os valores dos registradores, e contexto de *software* como lista de arquivos abertos, dados das variáveis e endereçamento de memória, entre outros. Essas informações devem ser salvas em memória para o processo que está saindo de execução e carregadas da memória para o novo processo que entra em execução.

Threads, assim como processos, possuem contextos e revezam o tempo de execução com as demais CPUs. Um conjunto de *threads*, por compartilharem muitas informações em comum entre si (como endereçamento de memória), podem trocar de contexto umas com as outras com poucas mudanças em comparação à troca entre processos distintos.

Figura 1 – Representação da divisão dos sistemas multiprocessados



TEMA 2 – PROBLEMAS DE CONCORRÊNCIA E SOLUÇÕES

De tão frequentes, alguns problemas recorrentes dentro da programação paralela/concorrente acabaram se tornando conceitos importantes que devem ser analisados e evitados.



É importante começar este estudo trazendo o conceito de concorrência e paralelismo que, embora semelhantes, não são a mesma coisa. Processos concorrentes são aqueles que executam em um mesmo período de tempo dividindo recursos de memória e CPU; os processos se alternam entre quais estão ativos e quais estão dormentes. Por exemplo, os computadores com um único processador operam múltiplos processos de forma concorrente alternando o tempo de processador dedicado a cada um. Essa alternância ocorre tão rápido que temos a impressão de que operam de forma simultânea.

O paralelismo, por outro lado, seriam processos executando simultaneamente. Múltiplos núcleos podem executar diferentes processos não somente ao mesmo tempo como também ao mesmo instante.

Threads concorrentes que acessam uma mesma região compartilhada da memória podem gerar situações problemáticas de sincronismo dos dados. Um exemplo clássico é a chamada *condição de corrida*.

2.1 Condição de corrida

A condição de corrida é uma falha que ocorre quando uma entidade faz a leitura de uma informação para realizar uma ação e no meio-tempo entre a leitura da informação e a tomada dessa ação a informação original muda por conta de outra entidade e se torna obsoleta. Esse problema ocorre dentro dos mais distintos contextos de *hardwares* e *softwares*. No nosso contexto de uma aplicação *multithreading*, um exemplo clássico é o da escrita em um arquivo. Suponha o seguinte código:

```
if (ArquivoExiste("arquivo.txt")) {  
    EscreverDados("arquivo.txt");  
}
```

Primeiro verificamos se o arquivo existe, para então escrever alguma informação nele. Porém, se logo após verificarmos que o arquivo realmente *existe* e então, *antes* de enviarmos a informação, outra *thread* *deletar o arquivo*, então não teremos mais o nosso destino para os dados criados e uma falha ocorrerá. Dentro da analogia da corrida, a *thread* que chegou primeiro executou corretamente enquanto a seguinte falhou. Em resumo, a condição de corrida ocorre em programas cujo resultado final depende da ordem de execução das *threads*. Para evitarmos esse problema, temos de forçar uma sincronia e impor

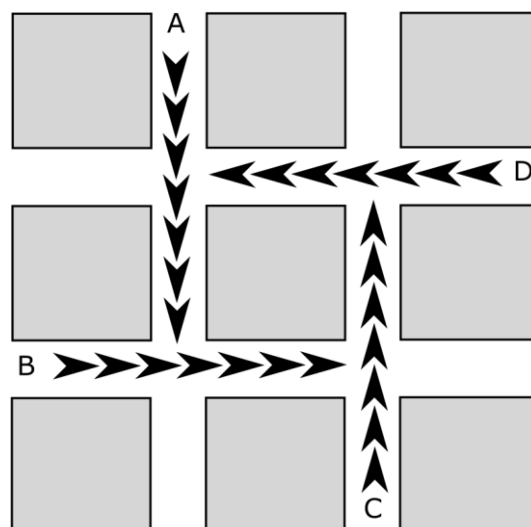


uma ordem de execução travando – do inglês *lock* – determinado recurso ou posição de memória para uma única *thread*.

2.2 Deadlock

O travamento de certas regiões de memória ou de certos recursos de sistema pode ocasionar outro problema conhecido pela expressão em inglês *deadlock*, que se traduz como *impasse*. Os *deadlocks* ocorrem quando uma *thread A* requisita o acesso e trava um determinado *recurso X*, e antes de liberar esse *recurso X* ele também requisita um *recurso Y*. Porém, uma *thread B* que travou o *recurso Y* por sua vez requisita o *recurso X*. A dependência aqui exemplificada considerou somente duas *threads* e dois recursos, porém, essa dependência cíclica pode envolver várias *threads* e recursos, gerando um impasse generalizado. Um exemplo fora da computação, mas que ilustra bem essa situação, é de um engarrafamento em um cruzamento. Na Figura 2, vemos um exemplo desta situação. O fluxo da pista A deseja cruzar a pista B, que está ocupada pelo fluxo da própria pista B, que deseja cruzar a pista C, que está ocupado pelo próprio fluxo da pista C, que deseja cruzar a pista D, que está ocupado pelo fluxo da pista D, que deseja cruzar a pista A, fechando o impasse cíclico.

Figura 2 – *Deadlock* no trânsito, as setas representam o fluxo de carros





```
1 void ProcessoA() {
2     travar(&recurso_1);
3     travar(&recurso_2);
4     usa_recursos();
5     liberar(&recurso_2);
6     liberar(&recurso_1);
7 }
8 void ProcessoB() {
9     travar(&recurso_2);
10    travar(&recurso_1);
11    usa_recursos();
12    liberar(&recurso_2);
13    liberar(&recurso_1);
14 }
```

Acima, o código exemplifica uma situação de *deadlock*. Imagine uma *thread* executando *ProcessoA* e outra executando o *ProcessoB*. Nas linhas 2-3 e 9-10, os *recurso_1* e *recurso_2* são travados em ordem trocada em cada processo. Caso simultaneamente as *threads* executem as linhas 2 e 9, ambas ficarão indefinidamente aguardando a liberação dos recursos que jamais irá ocorrer. A simples troca da ordem das linhas 9-10 resolveria esta situação, pois o *recurso_2* só poderia ser solicitado por quem já tivesse o *recurso_1*.

2.3 Livelock

O impasse causado pelo *deadlock* é facilmente identificado pelo fato das *threads* estarem travadas em modo de espera. No entanto, temos uma variante desse impasse conhecido pelo termo em inglês *livelock*, na qual os processos estão travados, porém operantes, e sincronizadamente entram em um ciclo no qual liberam e travam recursos sem ativamente realizar nenhum trabalho útil.

Um exemplo fora da computação que está presente no cotidiano na vida corriqueira seriam duas pessoas que se cruzam frente a frente em um corredor e sincronizadamente abrem espaço um para o outro permanecendo travados sem efetivamente progredirem. Em resumo, no *deadlock* as *threads* ficam travadas em estado dormiente, e no *livelock* estão travadas em estado ativo.

Essa situação pode acontecer quando dois (ou mais) processos desejam utilizar dois recursos simultaneamente. Um processo solicita o primeiro recurso e quando solicita o segundo recurso este já foi solicitado por outro processo, que também está aguardando o primeiro recurso da mesma forma gerando um ciclo. Os processos não estão propriamente travados, mas sim operantes e constantemente conferindo a liberação do recurso aguardado.



2.4 Starvation

Outro problema que pode acontecer quando temos uma política de escalonamento de recursos que deixa um conjunto de processos com menor prioridade em relação a outro conjunto é chamado no inglês de *starvation*, que se traduz como inanição. Suponha um processo de baixa prioridade em um cenário de elevada demanda por um recurso: os processos com alta prioridade sempre irão receber o recurso primeiro, de forma que o processo de baixa prioridade ficará aguardando sem garantias de que receberá o recurso em algum momento, morrendo de fome.

Um exemplo do cotidiano da vida seria uma fila qualquer (banco, caixa lotérica, mercado) com acesso prioritário (idosos, gestantes e pessoas com deficiência). Quando um cliente prioritário termina de ser atendido, outro chega, e antes que esse termine seu atendimento outro chega, e depois mais outro, e assim sucessivamente, de forma que o cliente sem acesso prioritário aguardará indefinidamente.

Em linhas gerais a solução seria incrementar gradualmente a prioridade dos processos que aguardam por muito tempo.

2.5 Mutex

Quando temos múltiplas *threads* querendo permissão de escrita em uma mesma posição de memória, a solução amplamente adotada para evitar a condição de corrida é a técnica chamada de *mutex*, do inglês *mutual exclusion*, traduzido como exclusão mútua. A solução consiste de um marcador único, e quem possui esse marcador pode escrever e os demais aguardam sua vez para receber o marcador. Um exemplo na vida cotidiana seria a chave de um banheiro: quem possui a chave tem acesso. Neste exemplo a chave (*mutex*) confere acesso ao banheiro (recurso, variável) para uma única pessoa (*thread*) por vez, e quando a pessoa termina de utilizar passa a chave para a próxima pessoa (outra *thread*) na fila.

Nas linguagens C/C++ temos diversas bibliotecas que implementam *threads*. Destacam-se pthread, openMP e boost-thread; todas implementam *mutex* que variam em sintaxe, mas que operam da mesma forma. Abaixo descrevemos duas operações comuns nessas bibliotecas:



- *lock*, permite que uma *thread* segure a *mutex* e garante que qualquer outra *thread* que execute a linha de comando em questão fique travada aguardando a liberação da *mutex* passada por parâmetro.
- *unlock*, destrava o *mutex* passado por parâmetro permitindo outras *threads* a acessem.

```
void funcao_multi_thread(){  
    Mutex m;  
    mutex_lock(m);  
    //Operações críticas  
    mutex_unlock(m);  
}
```

2.6 Semáforo

Os semáforos funcionam de forma semelhante ao *mutex*, porém, permitem um número pré-determinado de *threads* executarem simultaneamente uma mesma região crítica. Um exemplo do dia-a-dia seria um restaurante: existe um número limitado mesas (recursos) e quando uma pessoa (*thread*) libera uma mesa, abre espaço para outra *thread* que esteja na fila utilizar. Sendo assim, a diferença para o *mutex* é a existência desse “número de vagas”. Dentro das várias bibliotecas que implementam *threads*, o controle é feito por meio de dois comandos. A sintaxe varia um pouco, mas a estratégia é a mesma:

- *wait()* Traduz-se como *espera*. É o comando que vem antes da região crítica, tal qual como a fila do restaurante. Se não tiver ninguém na sua frente e tiver vaga, você entra no restaurante (executa a região crítica), do contrário, você espera.
- *signal()* Traduz-se como *sinal*, é o comando que sinaliza que a *thread* está deixando a região crítica (liberando uma vaga no restaurante).

No exemplo abaixo, em um código em uma linguagem hipotética, mostramos como seria a implementação para um semáforo com 4 vagas. O comando *wait* e *signal* cuida sozinho da gestão deste valor.



```
void funcao_multi_thread(){
    Semaforo s = 4;
    semaforo_wait(s);
    //Operações críticas
    semaforo_signal(s);
}
```

Um semáforo de valor 1, também chamado de semáforo binário, é idêntico em funcionamento ao *mutex*.

2.7 Monitor

Algumas linguagens de programação – damos destaque aqui para o Java – implementam de forma natural o conceito de exclusão mútua via monitores. É o exato mesmo conceito que o *mutex*: você pode aplicar para manipulação de um objeto em um dado momento ou para um método inteiro. No Java, a palavra chave que executa monitor é *synchronized*.

```
public void metodo_multi_thread() {
    synchronized(algumObjeto) {
        // área crítica
    }
}
public synchronized void metodo_multi_thread() {
    //Todo o código é crítico
}
```

No código acima, no primeiro método *algumObjeto* só pode ser acessado por outras *threads* depois que aquela área crítica finalizar. E no segundo método pode ser executado por uma única *thread* por vez.

TEMA 3 – JANTAR DOS FILÓSOFOS

Para ajudar aprofundar nosso conhecimento em concorrência de múltiplas *threads* ou processos, vamos estudar alguns problemas que são considerados clássicos e assim nos ajudar a identificar suas manifestações em nossas aplicações. O primeiro dele será o Jantar dos Filósofos.

Suponha que temos uma mesa redonda na qual estão jantando 5 filósofos. À frente de cada filósofo há um prato, na sua esquerda e na sua direita há 1 garfo de cada lado – ao todo 5 garfos, ou seja, o garfo direito de um filósofo é o garfo esquerdo do filósofo da direita. A Figura 3 ilustra a situação. Quando



um filósofo não está comendo ele está pensando. No momento que um filósofo decide comer, ele pega primeiro o garfo da sua esquerda e depois o garfo da sua direita, e só comerá utilizando os dois garfos simultaneamente. O jantar procede sem problemas até o momento que simultaneamente todos os filósofos decidem comer ao mesmo tempo. Todos pegam o garfo da esquerda e nenhum consegue pegar o garfo da direita e ficam em uma situação de impasse.

O problema do jantar dos filósofos pode acontecer dentro de diversos contextos da computação; no caso de aplicações *multithread*, várias *threads* podem se bloquear simultaneamente de forma cíclica como no problema do *deadlock*.

Existem diversas soluções para esse problema, e a mais simples é ilustrada abaixo. Ela envolve simplesmente modificar a ordem em que se pega os garfos. Veja o código em linguagem hipotética abaixo que ilustra esta situação:

```
void filosofo_problema(int i){
    Mutex mut[5];
    while(true){
        pense();
        int proximo_garfo = (i+1) mod 5;
        mutex_lock(mut[i]);
        mutex_lock(mut[proximo_garfo]);
        comer();
        mutex_unlock(mut[i]);
        mutex_unlock(mut[proximo_garfo])
    }
}
```

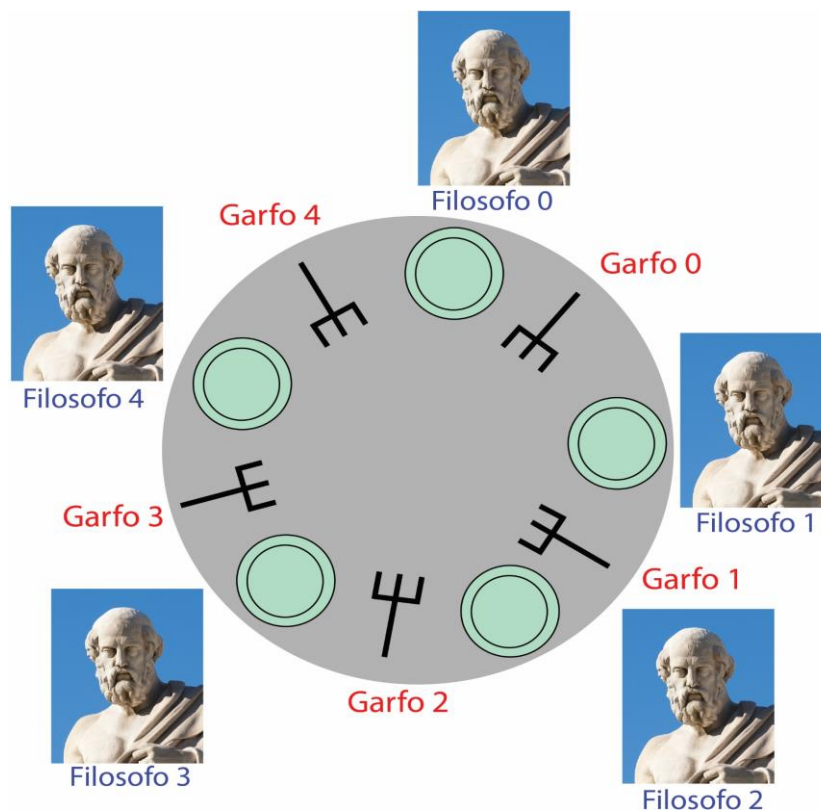
Na versão acima caso simultaneamente sejam requisitados todos os garfos, todos os filósofos (*threads*) ficarão aguardando a liberação do segundo garfo por tempo indeterminado.



```
void filosofo_correto(int i){  
    Mutex mut[5];  
    while(true){  
        pense();  
        int proximo_garfo = (i+1) mod 5;  
        mutex_lock(mut[min(i,proximo_garfo)]);  
        mutex_lock(mut[max(i,proximo_garfo)]);  
        comer();  
        mutex_unlock(mut[i]);  
        mutex_unlock(mut[proximo_garfo])  
    }  
}
```

Nessa segunda versão, o problema é corrigido simplesmente modificando a ordem de travamento dos recursos: os dois filósofos que utilizam o garfo de $id=0$ priorizam esse garfo. Dessa forma, não vão travar outro recurso sem ter o garfo 0 liberado antes, evitando assim o efeito de travamento em cadeia.

Figura 3 – Jantar dos filósofos



Crédito: Vangelis/Shutterstock.



TEMA 4 – PRODUTOR E CONSUMIDOR

O segundo problema que vamos analisar do ponto vista de concorrência é o problema do produtor e consumidor.

O problema de sincronização do produtor/consumidor consiste de dois grupos de *threads*:

- *Produtoras*, que inserem dados em um *buffer* de tamanho limitado, uma por vez.
- *Consumidoras*, que consomem dados do mesmo *buffer*, uma por vez também.

O desafio do problema está em evitar que as produtoras escrevam no *buffer* quando ele estiver cheio (*overflow*), e que as consumidoras tentem ler o *buffer* quando ele estiver vazio (*underflow*).

```
1 Semaforo sem_preencher = 0; // evitar underflow
2 Semaforo sem_esvaziar = TAMANHO_BUFFER; // evitar overflow
3 void produtor() {
4     while (true) {
5         item = produzir_item();
6         semaforo_wait(sem_esvaziar);
7         inserir_buffer(item);
8         semaforo_signal(sem_preencher);
9     }
10 }
11 void consumidor() {
12     while (true) {
13         semaforo_wait(sem_preencher);
14         item = remover_item_buffer();
15         semaforo_signal(sem_esvaziar);
16         consumir_item(item);
17     }
18 }
```

A ideia desta solução é manter dois semáforos, um para controlar o *overflow* (linha 1) e outro o *underflow* (linha 2).

Produtor:

- Aumenta o semáforo que conta os preenchidos (linha 6)
- Diminui o semáforo que conta os espaços vazios (linha 8)

Consumidor:

- Diminui o semáforo que conta os preenchidos (linha 13)
- Aumenta o semáforo que conta os espaços vazios (linha 15)



Dessa forma, utilizamos dois semáforos para evitar duas situações indesejadas (*overflow* e *underflow*). Esse problema se aplica em diversos contextos, por exemplo, em um jogo de videogame podemos ter uma *thread* lendo comandos que o jogador executa no teclado/controle (produtora) enquanto outra *thread* processa esses comandos e gera uma resposta (consumidora). Se o jogador executar comandos muito rapidamente, eles serão salvos em um *buffer*, não sendo perdidos.

TEMA 5 – BARBEIRO DORMINHOCO

E fechando a análise de alguns dos problemas clássicos de concorrência, vamos nos focar no problema do barbeiro dorminhoco.

O problema do barbeiro dorminhoco se repete em diversos contextos dentro da computação – é outro problema clássico de sincronização. Ele é apresentado pela seguinte analogia. Suponha uma barbearia com um único barbeiro sonolento que passa pelas seguintes situações:

- *Não tem clientes*: o barbeiro dorme (linha 8);
- *Um cliente chegou e o barbeiro dorme*: o cliente acorda o barbeiro (linha 22), que o atende. Na sequência, o barbeiro verifica se existe algum cliente na sala de espera (linha 8). Se sim, faz o atendimento; do contrário volta a dormir;
- *Um cliente chegou e o barbeiro está ocupado*: ele aguarda o barbeiro na sala de espera (linha 24);
- *Um cliente chega e a sala de espera está cheia*: ele vai embora (linha 27).

A seguir o código comentado equivalente que soluciona o problema:

```
1 Mutex barbeiroAcordado; //Indica se o barbeiro está acordado
2 Mutex cadeira; //Cadeira livre
3 Semaforo salaEspera = 0; //Clientes na salaEspera
4 int assentosLivres = N; //Assentos livres na sala de espera
5
6 void barbeiro() {
7     while (true) {
8         sem_wait(salaEspera);
9         mutex_lock(cadeira);
10        assentosLivres += 1;
11        mutex_unlock(barbeiroAcordado);
12        mutex_unlock(cadeira);
13        // barbeiro faz o corte de cabelo aqui
14    }
15 }
16
```



```
17 void cliente() {
18     while (true) { //Emulando infinitos clientes
19         mutex_lock(cadeira);
20         if (assentosLivres > 0) { //Existem assentos livres
21             assentosLivres -= 1;
22             sem_signal(sala_espera);
23             mutex_unlock(cadeira);
24             mutex_lock(barbeiro); //Espera o barbeiro
25             //cliente tem o cabelo cortado aqui
26         }
27         else { //Não tem assentos livres
28             mutex_unlock(cadeira);
29             //Sair sem cortar o cabelo
30         }
31     }
32 }
```

Existem muitas soluções para esse problema. Na implementação apresentada aqui, o elemento chave é o *mutex* cadeira (linha 2), que garante que apenas uma *thread* irá modificar o estado por vez, tanto da parte do barbeiro (linha 9) quanto dos clientes (linha 19). Um exemplo proeminente na computação seria o trabalho de uma impressora: na analogia, ela é o barbeiro que dorme, ficando em *stand-by*, e quando uma solicitação de impressão (cliente) chega via rede ela atende, coloca na fila, ou informa que está sobrecarregada, caso a fila esteja cheia.

FINALIZANDO

Nesta aula finalmente colocamos nossas mãos nos problemas computacionais e suas soluções envolvendo sincronia em um ambiente concorrente entre *threads* ou processos. Optamos por não focar em nenhuma linguagem ou biblioteca específica para que o aluno possa abstrair essas soluções para qualquer contexto. Existem uma diversidade de problemas relevantes envolvendo sincronia e concorrência; focamos aqui em três dos principais que ilustram situações muito comuns que podem ser abstraídas para outras diversas situações.